1 Decision Tree - Theorem 8.1 Lesson

1.1 1. The Decision Tree and Permutations

For a **comparison sort** on N elements, every possible outcome (or permutation) of the input can be represented as a *leaf* in a *decision tree*.

- **Permutations**: For N elements, there are $N!$ possible ways to arrange them, so there are $N!$ reachable leaves in the decision tree. Sorting N elements requires handling all these permutations, demonstrating that sorting is fundamentally about managing permutations, not just iteration.

1.2 2. $N!$ and Stirling's Approximation

When dealing with the factorial $N!$, its growth is *super-exponential*, making direct calculation of comparison costs difficult for large N.

- **Stirling's Approximation** helps simplify $N!$ by approximating it with a logarithmic term, $N \log N$. This approximation is essential for deriving bounds on the number of comparisons in a sort and allows the use of logarithms in the proof.

1.3 3. Logarithms and the Height of the Decision Tree

The height h of the decision tree defines the worst-case number of comparisons in a sorting algorithm.

- The number of reachable leaves in the tree is at least $N!$, and the number of leaves in a binary tree of height h is at most 2^h. This gives the inequality:

$$N! \leq L \leq 2^h$$

- **Taking logarithms**: Applying logarithms to both sides of the inequality simplifies it to:

$$h \geq \log_2(N!)$$

  which provides a lower bound on the number of comparisons based on the height of the decision tree.

1.4 4. Theorem 8.1 and Asymptotic Optimality

- **Theorem 8.1** states that any comparison sort algorithm requires $\Omega(N \log N)$ comparisons in the worst case. This lower bound applies to any algorithm that uses comparisons to sort, such as heapsort or mergesort.
- **Heapsort and Mergesort** are *asymptotically optimal* comparison sorts because they achieve the lower bound $\Omega(N \log N)$ in both worst-case and average-case scenarios. Their upper bound is also $O(N \log N)$, meaning the worst-case number of comparisons matches the lower bound, making them optimal within this class.

1.5 5. Understanding Bounds

- **Bounds** refer to the *growth rate* of an algorithm's time complexity, which reflects how the algorithm's runtime increases as the input size N grows.
- The lower bound (Ω) tells us the minimum number of comparisons needed in the worst case, while the upper bound (O) gives us the maximum number of comparisons.
- In the case of heapsort and mergesort, their time complexity is tightly bound between $\Omega(N \log N)$ and $O(N \log N)$, making them efficient and optimal for sorting.

1.6 Summary of Theorem 8.1 Concepts

- **Decision Trees**: Represent the sorting process as a set of comparisons, with $N!$ permutations corresponding to the leaves.
- **$N!$ and Stirling's Approximation**: Simplify the factorial growth into $N \log N$ for easier analysis.
- **Logarithms**: Used to analyze the height of the decision tree and establish the worst-case lower bound on comparisons.
- **Asymptotic Optimality**: Heapsort and mergesort meet the $\Omega(N \log N)$ bound, making them optimal comparison sorts.
- **Bounds**: Refer to how the runtime of an algorithm grows with the size of the input, important for analyzing sorting efficiency.